
TB 2026

Aluno: Filipe Eismann¹, Gabriel Tondolo² e Vinícius Rocca³

Data: 11/06/2026

Matrícula: 2021520042¹, 2023520204² e 2020201189³

Curso: Engenharia de Computação^{1,2,3}

Multiplicador em Ponto Flutuante 32 bits

Resumo

Este trabalho apresenta o projeto e a implementação ASIC de um multiplicador em ponto flutuante de 32 bits conforme o padrão IEEE 754, desenvolvido em VHDL com arquitetura em pipeline de três estágios (pré-normalização, multiplicação das mantissas e pós-normalização) e validado por simulações RTL, síntese lógica e síntese física utilizando as ferramentas *Xcelium*, *Genus* e *Innovus* da *Cadence*. O circuito detecta e trata corretamente todas as exceções previstas pelo padrão (*overflow*, *underflow*, infinito e *Not a Number*), com funcionamento confirmado nas três etapas de verificação. Dentre os resultados mais significativos, destaca-se a obtenção de uma frequência máxima de operação de 368 MHz com *timing slack* positivo tanto na síntese lógica quanto no layout, e a potência dinâmica total de aproximadamente 2360 μW , valor inferior aos 11201 μW e 11054 μW reportados na literatura para implementações exata e aproximada em processo 90 nm, evidenciando o ganho proporcionado pela tecnologia de 45 nm adotada neste trabalho.

Palavras-chave: *Ponto flutuante IEEE 754, Síntese ASIC, Multiplicador.*

1. INTRODUÇÃO

A crescente demanda por capacidade computacional em aplicações de inteligência artificial e processamento de sinais digitais tem impulsionado o desenvolvimento de circuitos aritméticos de alto desempenho. O padrão IEEE 754 define a representação e as operações sobre números de ponto flutuante em sistemas digitais; na precisão simples (32 bits), um nú-

mero é composto por bit de sinal, expoente com viés de 127 (8 bits) e mantissa (23 bits) [1]. A multiplicação em ponto flutuante é particularmente relevante por sua frequência em *kernels* de álgebra linear e redes neurais profundas, onde operações de ponto flutuante respondem por cerca de 75 % do consumo de potência do núcleo do processador em aplicações intensivas [2]. A migração de cargas de inferência de redes neurais para dispositivos de borda (*edge AI*) intensifica a necessidade de unidades aritméticas eficientes em área e potência [3]. Nesse contexto, o fluxo de projeto ASIC (*Application Specific Integrated Circuit*) é a abordagem mais adequada, pois permite controle preciso sobre os parâmetros de Potência, Desempenho e Área (PPA) por meio de síntese lógica, posicionamento, roteamento e verificação física sistemática [4].

A multiplicação de dois operandos em ponto flutuante de precisão simples compreende três etapas fundamentais executadas sobre os campos individuais de cada operando [1]. Primeiramente, o bit de sinal do resultado é obtido pela operação *XOR* entre os bits de sinal dos dois operandos. Em seguida, os expoentes são somados e o viés de 127 é subtraído do resultado, produzindo o expoente final. Por último, e mais custosa em termos de área e latência, ocorre a multiplicação das mantissas de 24 bits — cada uma composta pelo bit implícito “1” concatenado aos 23 bits armazenados. Esse produto gera um resultado de 48 bits que deve ser normalizado e arredondado antes de ser truncado aos 23 bits da mantissa de saída [2]. Além dessas etapas, o circuito deve detectar e tratar as exceções previstas pelo padrão: *overflow*, quando o expoente excede o valor máximo representável; *underflow*, quando o resultado é pequeno demais para ser normalizado; operação inválida, para entradas como *NaN*; e divisão por zero, para operandos nulos. A etapa de multiplicação das mantissas é o principal gargalo de desempenho, motivando o uso de algoritmos como a codificação de *Booth Radix-4*, que reduz o número de produtos parciais à metade, e a árvore de *Wallace*, que comprime esses produtos parciais em apenas dois vetores para uma soma final com adicionador rápido [2].

2. REVISÃO DA LITERATURA

Aditya et al. [5] propõem uma abordagem aproximada utilizando Matemática Védica para um multiplicador de ponto flutuante em processo 90nm, de 32 bits, cujo fluxo de desenvolvimento ASIC foi realizado na ferramenta da *Synopsys*, e a compara com uma abordagem de multiplicação exata no mesmo processo. Anuradha et al. [6] apresentam uma implementação ASIC de um multiplicador eficiente em *area-delay* a partir de 3 unidades de processamento (*Sign Bit Generation Unit*, *Exponent Generation Unit* e *Mantissa Generation Unit*), em processo TSMC de 65nm. Este trabalho tem como foco a implementação de um multiplicador de ponto flutuante para utilização em sistemas VLSI de alta performance.

Por fim, este trabalho é baseado no *design* de uma FPU (*Floating Point Unit*) proposta por Jidan Al-eryani [7], com arquitetura de *pipeline*, disponível no portal *OpenCores*.

3. CIRCUITO

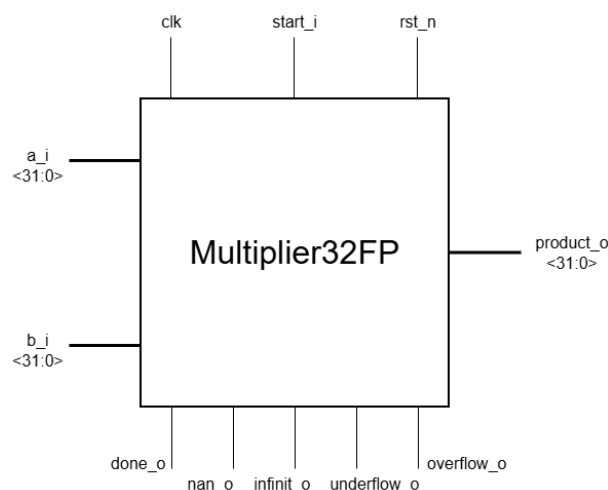
O multiplicador de ponto flutuante de 32 bits foi desenvolvido seguindo estritamente as especificações do padrão IEEE 754 para precisão simples. Para conciliar a alta taxa de processamento (*throughput*) exigida por aplicações modernas, como inferência em redes neurais na borda, e a temporização do circuito, adotou-se uma arquitetura baseada em *pipeline*.

A divisão do circuito em estágios de *pipeline* permite o paralelismo temporal das operações, reduzindo o caminho crítico e viabilizando o funcionamento estável do circuito em frequências de *clock* mais elevadas, conforme analisado nas seções subsequentes. O *design* tomou como referência a Unidade de Ponto Flutuante (FPU) proposta por Jidan Al-eryani [7].

3.1. Diagrama de topo VHDL

O esquemático de topo do multiplicador de ponto flutuante de 32 bits, que abstrai a complexidade interna do circuito, é mostrado na Figura 1. Este diagrama ilustra a interface do módulo `Multiplier32FP` com o sistema externo, definindo os sinais de entrada (*clock*, *reset*, dados de 32 bits *a_i* e *b_i*, e o sinal de controle *start_i*) e os sinais de saída (o produto de 32 bits *product_o*, o sinal de conclusão *done_o* e as *flags* de exceção). A Tabela 1 especifica textual e estruturalmente cada uma destas portas.

Figura 1: Esquemático de topo do multiplicador.

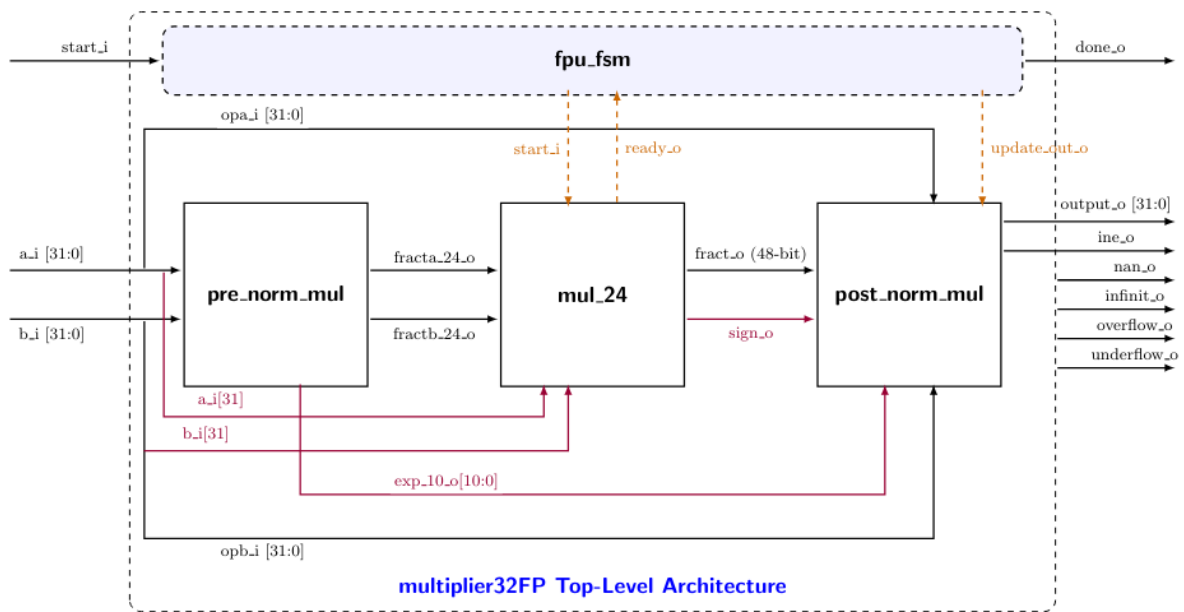


Fonte: Autor.

3.2. Arquitetura Interna e Estágios do Pipeline

Para realizar a multiplicação dos operandos de 32 bits, o *hardware* interno não opera em uma única etapa combinacional. Ele é segmentado em três estágios físicos de processamento sequencial (*pipeline*), conforme ilustrado na Figura 2. Esta arquitetura mitiga o acoplamento de caminhos críticos e viabiliza o fluxo de dados simultâneo sobre os campos estruturais do padrão IEEE 754.

Figura 2: Diagrama da arquitetura interna e estágios do *pipeline* Multiplier32FP.



Fonte: Autor.

A execução da multiplicação é dividida entre os seguintes módulos:

1. **Estágio 1 - Preparação (pre_norm_mul):** Responsável por desempacotar as entradas de 32 bits. Extrai os expoentes e as frações. Para a mantissa, o bit implícito é concatenado à esquerda da fração, gerando dois vetores operacionais de 24 bits. Simultaneamente, realiza a soma inicial dos expoentes e a subtração primária do viés (*bias*) de 127, repassando um expoente intermediário adiante.
2. **Estágio 2 - Execução Aritmética (mul_24):** Motor principal de cálculo. Recebe as mantissas preparadas e executa a multiplicação inteira, resultando em um produto bruto de 48 bits. Para paralelizar o cálculo e viabilizar altas frequências, emprega uma arquitetura *Dividir e Conquistar*: os vetores de 24 bits são subdivididos em blocos menores (12 bits e 6 bits). Múltiplos produtos parciais são calculados simultaneamente e acumu-

lados ao longo dos ciclos de *clock*. Em paralelo, este estágio resolve a polaridade do sinal através de uma porta *XOR* entre os bits mais significativos dos operandos originais.

3. **Estágio 3 - Formatação e Ajuste (*post_norm_mul*):** Os produtos do módulo precedente não estão adequados ao formato IEEE 754. Este módulo processa o resultado através de quatro sub-estágios lógicos internos:

- *Estágio 1:* Avalia o produto bruto para determinar a quantidade exata de deslocamento necessária para a normalização e realiza o ajuste preliminar do expoente.
- *Estágio 2:* Executa o deslocamento (*shift*) da mantissa para a esquerda ou direita, ativando uma *flag* caso ocorra perda de bits de precisão nas casas decimais.
- *Estágio 3:* Aplica as regras de arredondamento do padrão IEEE 754 (utilizando os bits descartados *Guard*, *Round* e *Sticky*) para reduzir a mantissa de volta a 23 bits, re-normalizando caso a operação gere um novo bit de *overflow*.
- *Estágio 4:* Avalia as condições geradas e empacota o resultado final de 32 bits, definindo inicialmente as *flags* específicos caso exceções tenham sido detectadas.

3.3. Detecção de Exceções e *Flags* de *Status*

A conformidade com o padrão IEEE 754 exige o tratamento de condições de contorno aritméticas. O módulo *post_norm_mul* avalia o resultado e repassa os dados para a entidade de topo (*Multiplier32FP*), onde as seguintes *flags* de *status* são exportadas:

- ***nan_o*** (*Not a Number*): Ativada em nível lógico alto quando a operação envolve uma indefinição matemática ou quando uma das entradas já é um *NaN*.
- ***infinitt_o***: Sinaliza que o resultado, ou um dos operandos, extrapolou os limites de representação ou que a operação resulta naturalmente em infinito.
- ***overflow_o***: Indica que a soma e ajuste dos expoentes ultrapassou o valor máximo representável para o formato de precisão simples ($E_{final} > 254$).
- ***underflow_o***: Ativada quando o expoente resultante é inferior ao limite mínimo passível de normalização, caracterizando um número subnormal.

3.4. Protocolo de Interface e Sincronização (FSM)

A interface do módulo é dada pelos sinais descritos na Tabela 1, presentes no módulo de topo (*FSM*).

Tabela 1: Descrição das portas do circuito

Nome	Tamanho	Tipo	Conexão	Descrição
clk	1	entrada	externa	Clock global do circuito
rst_n	1	entrada	externa	Reset global do circuito ativo em baixo
start_i	1	entrada	externa	Indica quando deve ser iniciada uma nova operação de multiplicação
a_i	32	entrada	externa	Dado a ser multiplicado
b_i	32	entrada	externa	Dado a ser multiplicado
product_o	32	saída	externa	Resultado da multiplicação
done_o	1	saída	externa	Indica que a multiplicação terminou e o valor na saída é válido
nan_o	1	saída	externa	<i>flag</i> para indicar que um operando que não é um número
infinif_o	1	saída	externa	<i>flag</i> para indicar que um operando é infinito
overflow_o	1	saída	externa	<i>flag</i> para indicar que o resultado da operação gerou <i>overflow</i>
underflow_o	1	saída	externa	<i>flag</i> para indicar que o resultado da operação gerou <i>underflow</i>

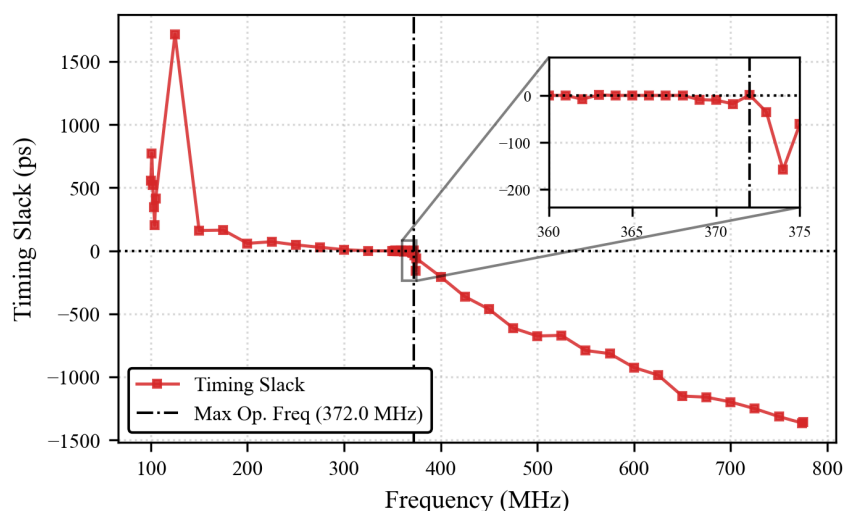
Fonte: Autor.

4. RESULTADO E DISCUSSÃO

4.1. Obtenção da frequência máxima de operação

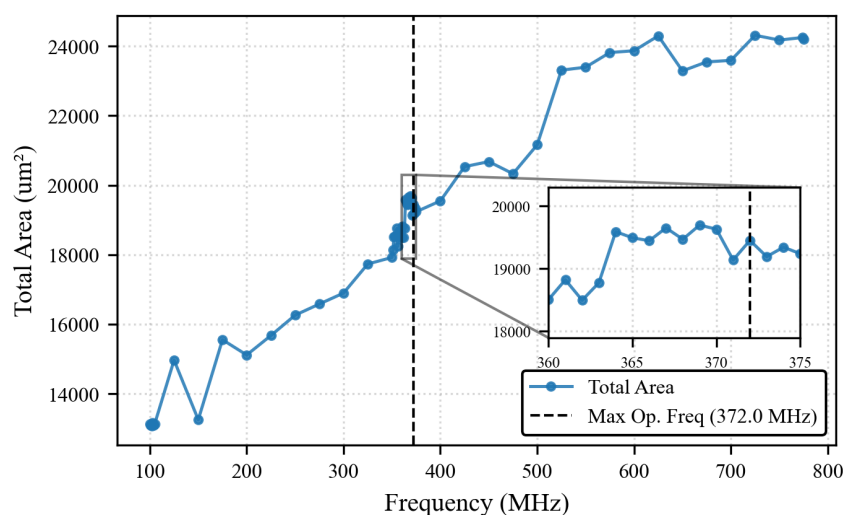
Para obter a máxima frequência de operação do circuito, foi realizada uma varredura de sínteses a nível de porta lógica até que a métrica de *timing slack* se tornasse negativa. A Figura 3 mostra o comportamento dessa métrica em função das frequências de *clock* na síntese. Podemos observar que, a partir da frequência de 365MHz, temos uma variação em torno de zero para o *slack*, o que indica que a ferramenta alterou células, área de circuito e caminhos críticos de modo a adequar o circuito a essas frequências. O gráfico da Figura 4 mostra que de fato há uma alteração significativa na área total entre cada passo de aumento de frequência.

Figura 3: Gráfico do *timing slack* em relação a frequência de síntese.



Fonte: Autor.

Figura 4: Gráfico da área total do circuito em relação a frequência de síntese.



Fonte: Autor.

Assim, foi determinado que a frequência máxima na síntese de portas lógicas é de 372MHz, que é a maior frequência de *clock* para qual o *slack* ainda é positivo. A frequência máxima que ainda apresentava *setup slack* maior do que zero para o *layout*, entretanto, foi de 371MHz.

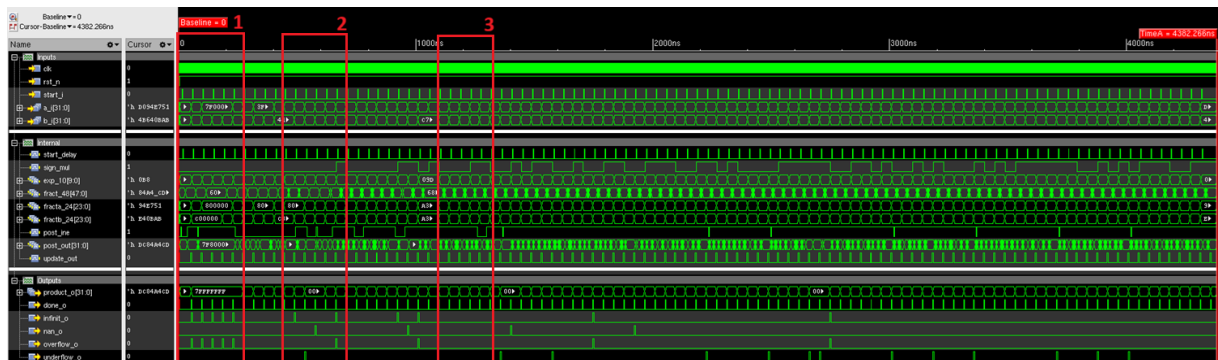
Para que a comparação entre síntese e *layout* fosse justa, foi reduzida a frequência a 368MHz, na qual ambas as etapas do desenvolvimento apresentam *slack* positivo.

4.2. Simulação sem SDF - RTL

Primeiramente, foi feita a simulação RTL (*Register Transfer Level*) considerando apenas o *test bench* com o vetor (100 operações), sem arquivo SDF (*Standard Delay Format*), na frequência máxima obtida (368MHz), utilizando a ferramenta *Xcelium* da *Cadence*. É possível observar que foi necessário cerca de 4400 ns para finalizar o processamento do vetor (Figura 5). Como se trata de uma simulação idealizada, sem detalhes de síntese física como capacitâncias parasitas e *delays*, os sinais se comportam exatamente como descrito no HDL.

Em todas as formas de onda a seguir, foram agrupados os sinais do módulo *DUV* (*Design Under Verification*), para fins de organização e melhor entendimento.

Figura 5: Simulação completa do vetor para a frequência máxima (368MHz). O detalhe 1 mostra operações com flag de *overflow* e *infinito* ativas, o detalhe 2 mostra resultados com *not a number* e *underflow* ativos, e o detalhe 3 mostra operações sem indicações de condição.

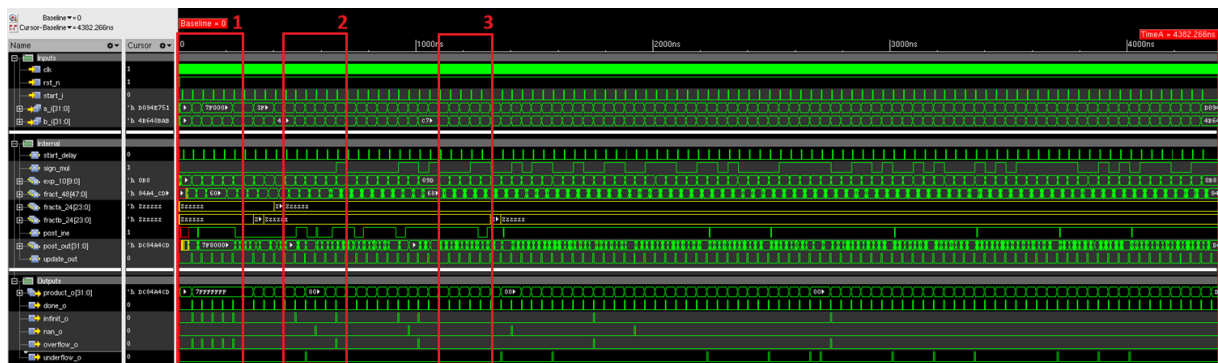


Fonte: Autor.

4.3. Simulação *netlist* com SDF - Síntese

Nesta etapa, foi realizada a simulação do *test bench* considerando o SDF anotado para a síntese lógica do circuito multiplicador, pré-*layout*. Na Figura 6 temos a simulação completa de todas as 100 operações do vetor, juntamente com os detalhes mostrados na subseção anterior. Aqui, existe a influência da biblioteca *worst* da tecnologia e dos arquivos LEF (*Library Exchange Format*) nas portas lógicas utilizadas.

Figura 6: Simulação completa do vetor para a frequência máxima (368MHz) com SDF anotado pré-layout e detalhes em evidência.



Fonte: Autor.

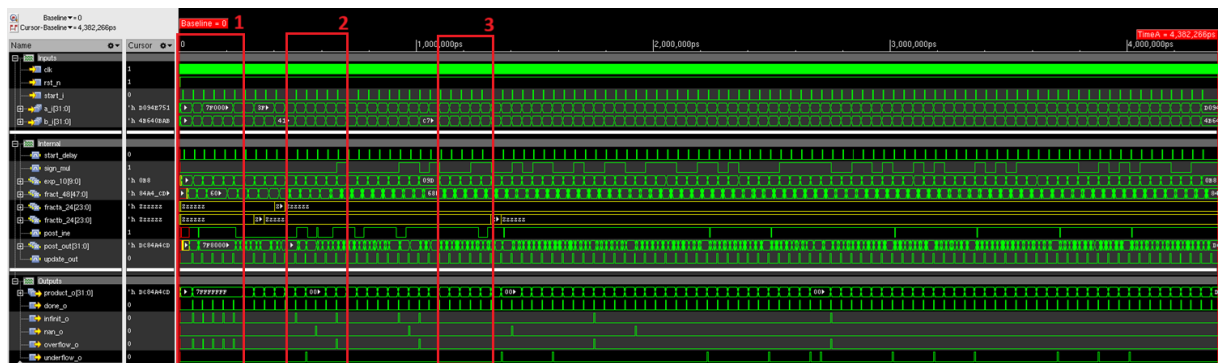
A principal diferença é visível nos sinais `fracta_24` e `fractb_24`, que possuem *bits* não utilizados nas operações, fazendo assim com que os sinais apresentem indicações de alta impedância durante todo o teste. Este comportamento ocorre devido a otimização das *nets* realizada na ferramenta de síntese, e não impacta o funcionamento correto do circuito, visto que são apresentados os mesmos resultados ao fim.

Dado que o *timing slack* é positivo para essa frequência, não há indicação de erro ou violações de temporização, apesar de ser necessário um tempo um ligeiramente mais elevado do que o RTL sem anotação para finalizar.

4.4. Simulação *netlist* com SDF - Layout

Considerando o *layout* sintetizado pela ferramenta *Innovus* da *Cadence* para a máxima frequência, temos a simulação do vetor completo mostrado na Figura 7. Os detalhes são os mesmos mostrados anteriormente. Aqui, temos o impacto completo estimado pela biblioteca *worst* e *LEF*, com as capacitâncias parasitas e *delays* de todas as *nets* e células.

Figura 7: Simulação completa do vetor para a frequência máxima com SDF anotado pós-*layout* (368MHz) com detalhes em evidência.



Fonte: Autor.

4.5. Comparação entre as simulações

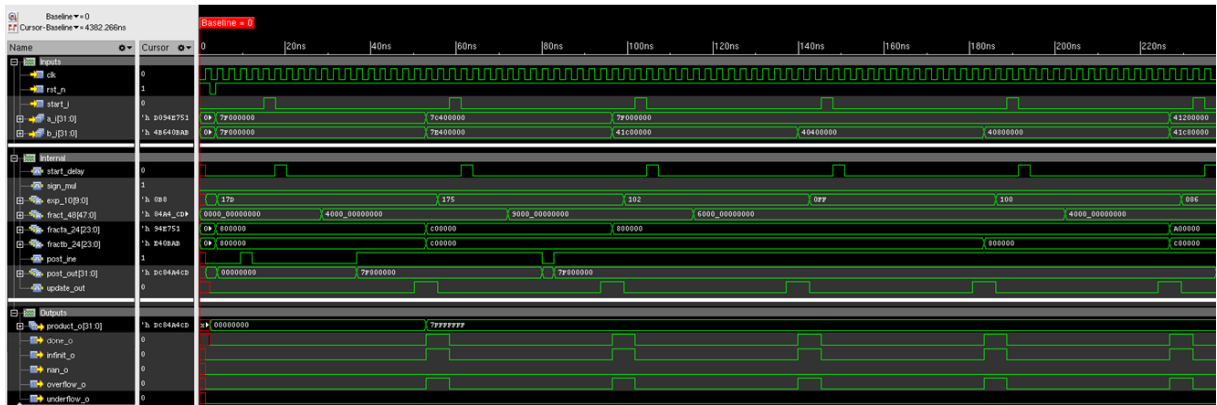
Nesta subseção, são feitas as comparações e comentários sobre cada um dos detalhes das simulações apresentadas previamente.

4.5.1 Detalhe 1 - Primeiras operações

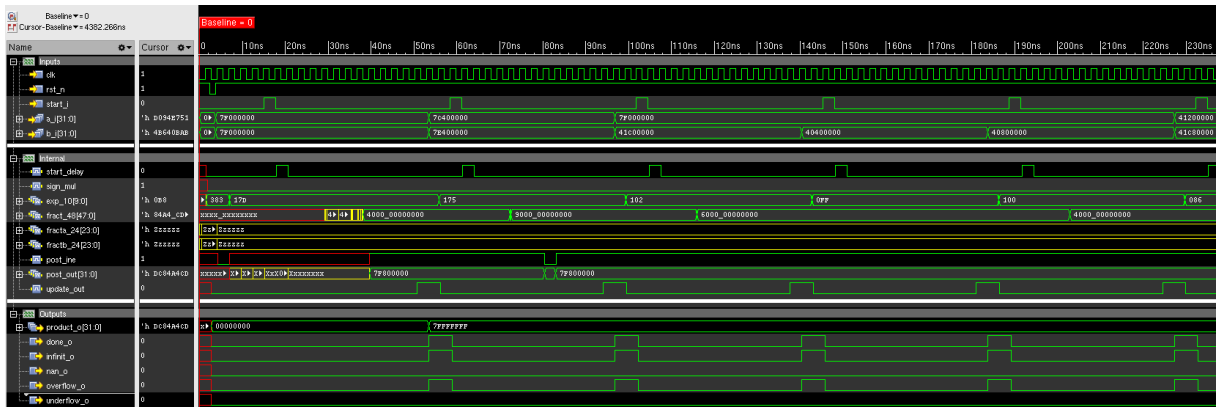
Na Figura 8a, temos o Detalhe 1 das primeiras operações da Figura 5, com os sinais de entrada, saída e internos. Nota-se que as primeiras 5 operações resultam em indicações de infinito (`infinitt_o`) e *overflow* (`overflow_o`). Na Figura 8b, temos o comportamento das operações do Detalhe 1 com SDF anotado a nível de síntese lógica. Aqui, observamos que o comportamento lógico é o mesmo, com as operações resultando nas mesmas *flags* e produto da simulação RTL. Na Figura 8c, temos o detalhe das primeiras operações para o SDF pós-*layout*. Pode-se observar que há mais tempo de indefinição no início para alguns sinais, devido ao maior *delay*, porém sem alterações na saída esperada.

Figura 8: Detalhe 1 das simulações.

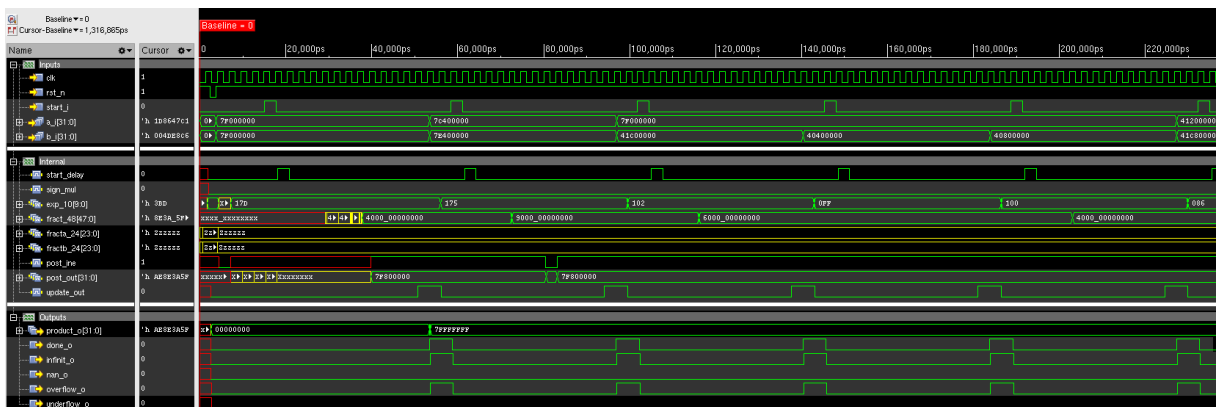
(a) RTL sem SDF.



(b) SDF Pré-layout.



(c) SDF Pós-layout.



Fonte: Autor.

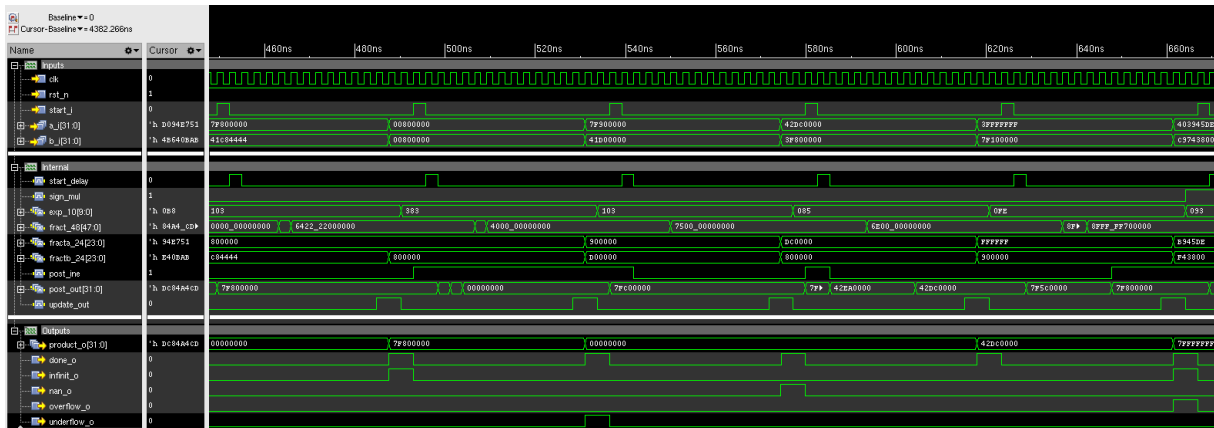
4.5.2 Detalhe 2 - Operações com *Not a Number* e *underflow*

o Na Figura 9a, é destacado o Detalhe 2 da simulação RTL. Aqui, temos operações com indicações de *Not a Number* e *underflow*. As Figuras 9b e 9c confirmam que o circuito

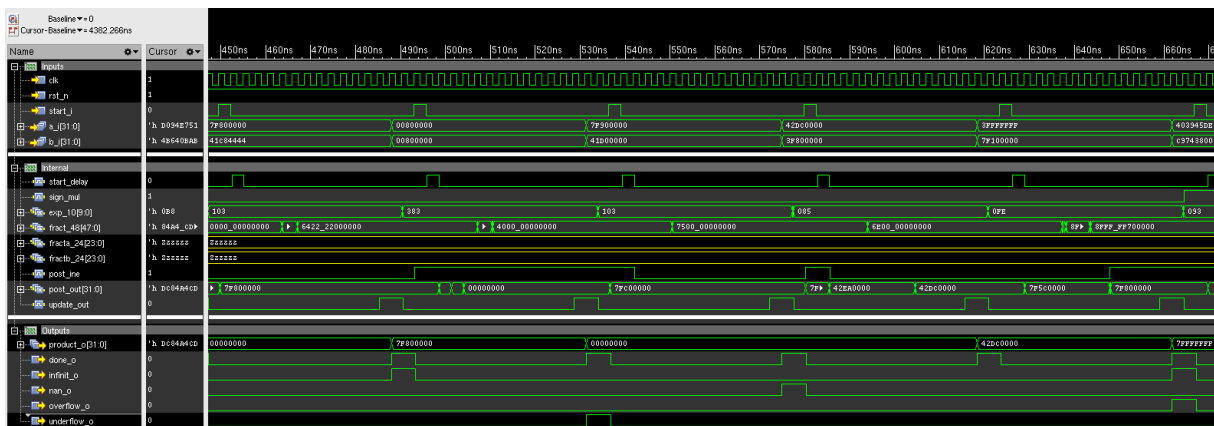
apresenta o comportamento lógico correto após a síntese e o *layout*, apesar de ter um pequeno *delay*.

Figura 9: Detalhe 2 das simulações.

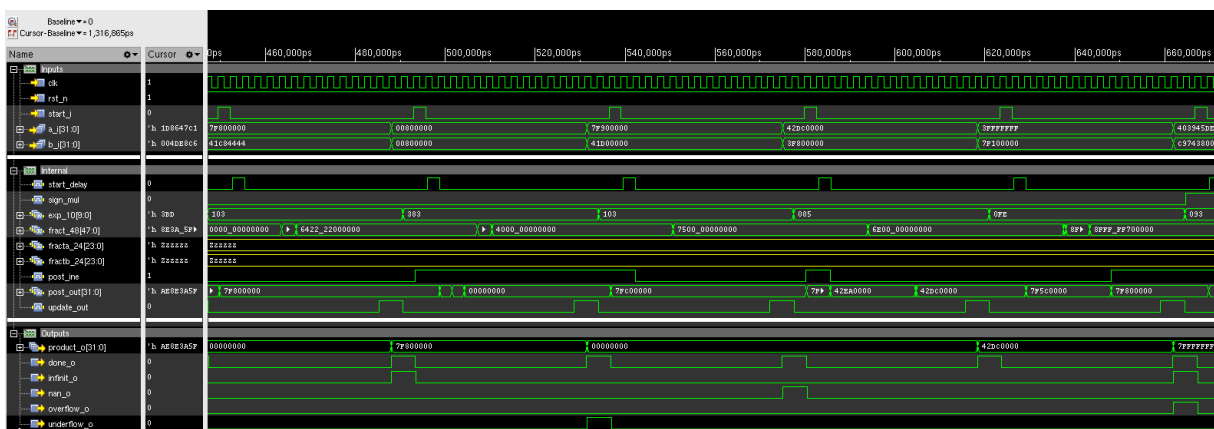
(a) RTL sem SDF.



(b) SDF Pré-*layout*.



(c) SDF Pós-*layout*.



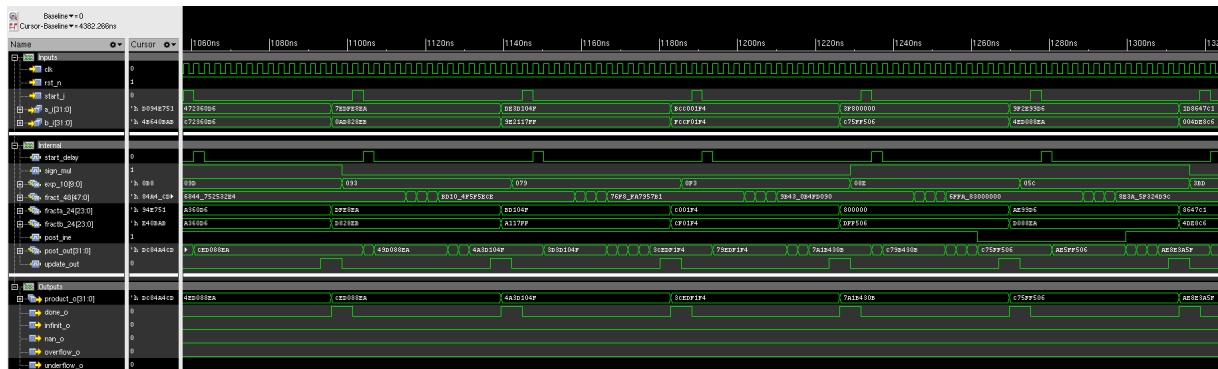
Fonte: Autor.

4.5.3 Detalhe 3 - Operações com saída válida (sem *flags*)

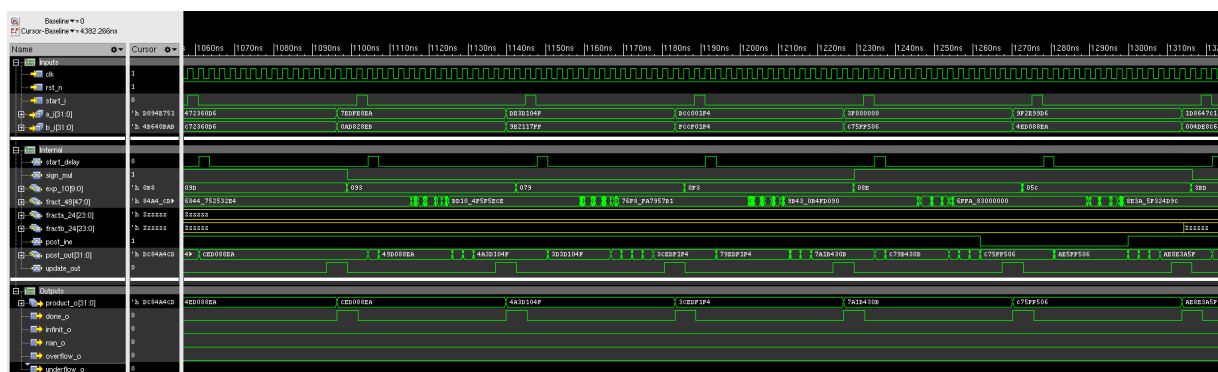
As operações seguintes (Figura 10a) apresentam resultados válidos para as multiplicações, mantendo o sinal de `done_o` alto por 2 bordas de *clock*. Novamente, o comportamento lógico da síntese (Figura 10b) e *layout* (Figura 10c) é correto e adequado ao RTL.

Figura 10: Detalhe 3 das simulações.

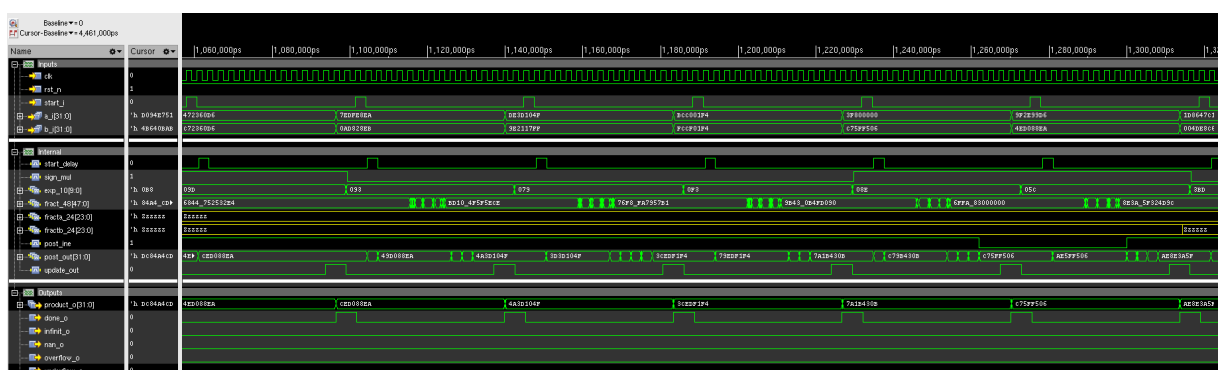
(a) RTL sem SDF.



(b) SDF Pré-*layout*.



(c) SDF Pós-*layout*.

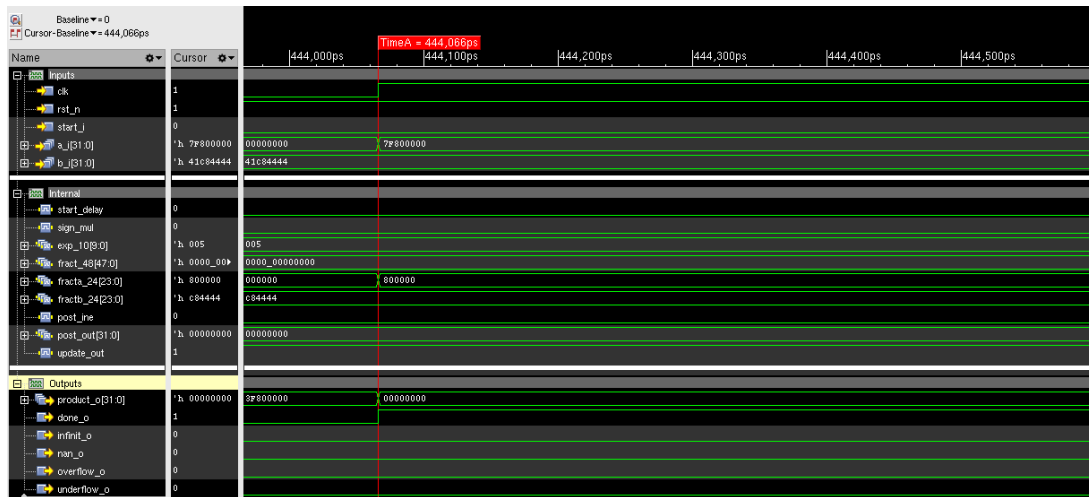


Fonte: Autor.

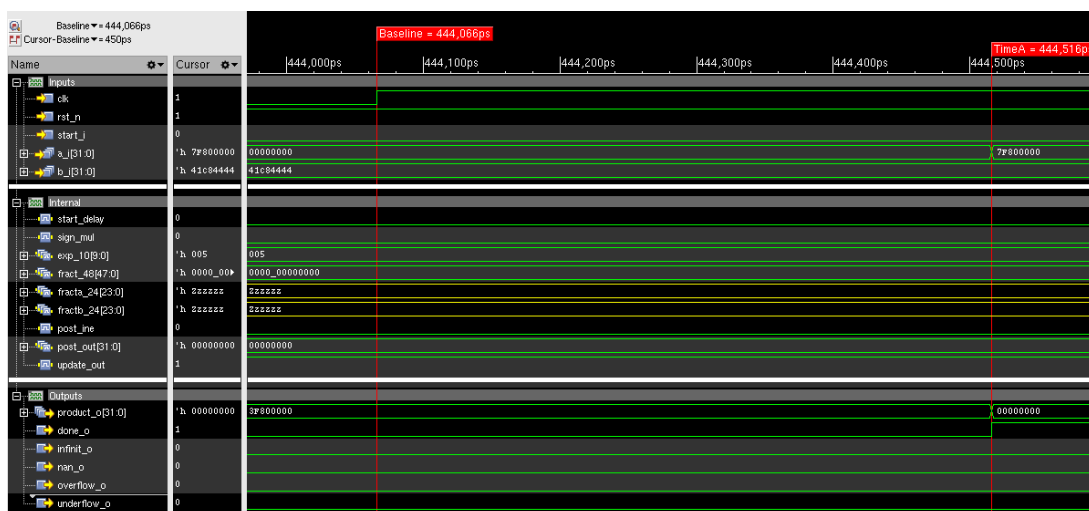
4.5.4 Delay de atualização da saída

Na Figura 11, podemos observar o *delay* entre a borda de *clock* e a atualização do sinal de saída para a síntese e *layout*. A simulação RTL sem SDF não apresenta *delay* entre o *clock* e a propagação dos sinais (Figura 11a), enquanto as *netlists* da síntese (Figura 11b) e *layout* (Figura 11c) apresentam atraso. Um detalhe importante é que, na simulação com SDF pós *layout*, o produto final leva um tempo maior para atualizar completamente após a subida do sinal *done_o*, o que gera uma leitura de falha no teste implementado no *test bench*, apesar de não haver violações de *constraints* de *timing*.

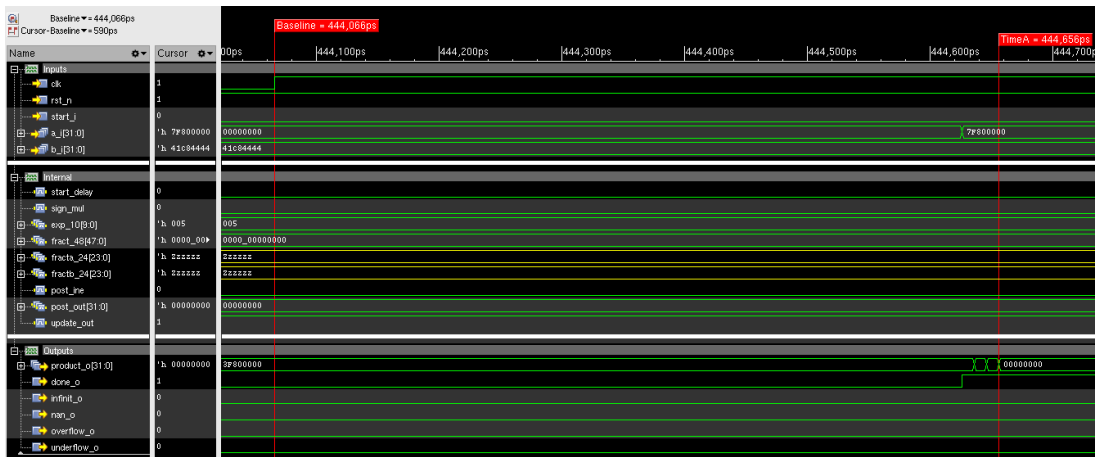
Figura 11: *Delay* entre o *clock* e a atualização do sinal de saída para cada simulação. O Marcação de *Baseline* está na borda de *clock* aos 444.066ns.



(a) RTL sem SDF



(b) SDF pré-layout

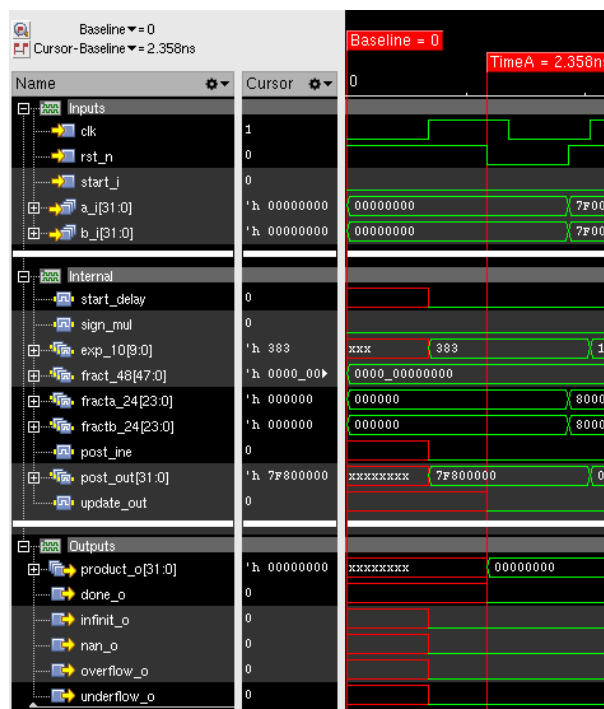


(c) SDF pós-layout

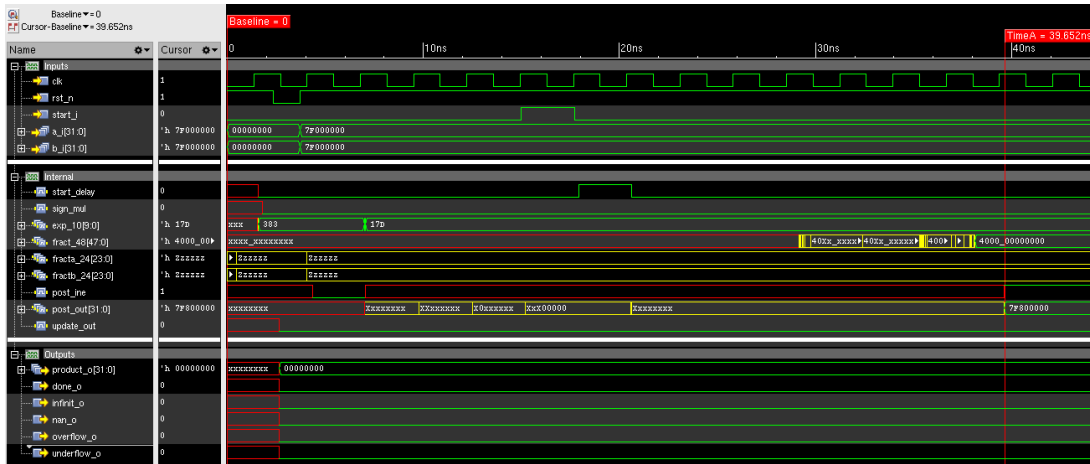
4.5.5 Tempo de Indefinição

Por fim, podemos observar o tempo de indefinição máximo dos sinais ao início de cada simulação na Figura 12. Novamente, o *delay* de caminho no *layout* é maior (Figura 12c), levando assim mais tempo para definir completamente os sinais utilizados no circuito.

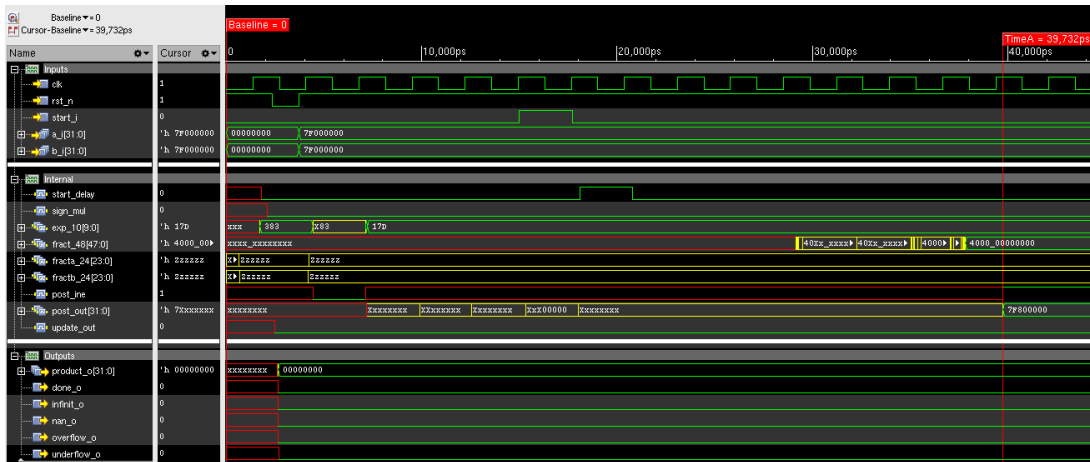
Figura 12: Tempos máximos de indefinição para cada simulação.



(a) RTL sem SDF



(b) SDF pré-layout



(c) SDF pós-layout

Na Tabela 2, temos as principais diferenças entre o *delay* e tempo de indefinição para cada simulação. Para 368 MHz, temos um *slack* de 0.01 ps, o que indica que o caminho crítico consome praticamente todo o período disponível, o que explica o maior tempo de indefinição pós-layout.

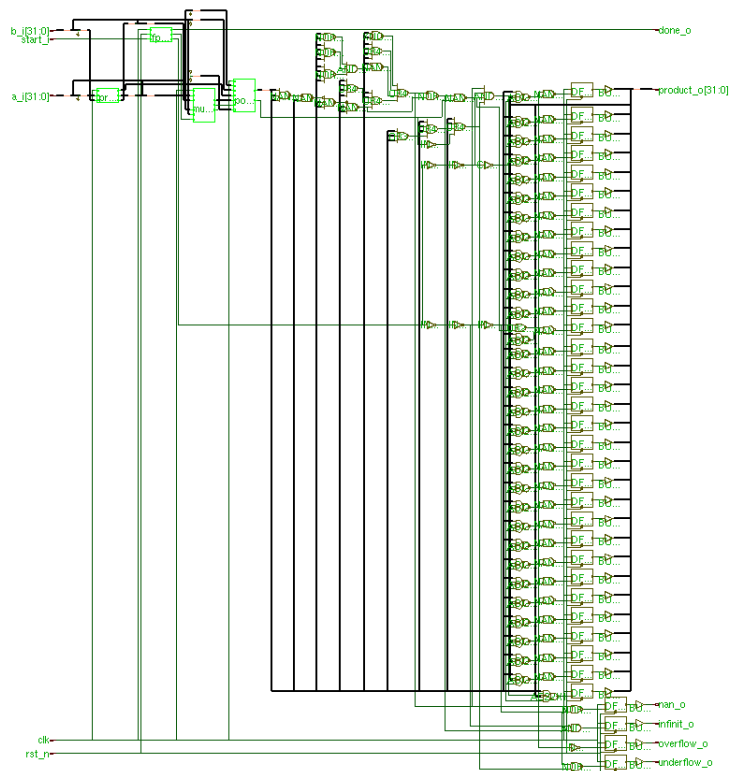
Tabela 2: Parâmetros de temporização para cada simulação

Parâmetro	RTL (sem SDF)	SDF Pré-layout	SDF Pós-layout
Delay clk -> product_o (ps)	0	450	590
Tempo Máximo de Indefinição (ns)	2.356	39.652	39.732

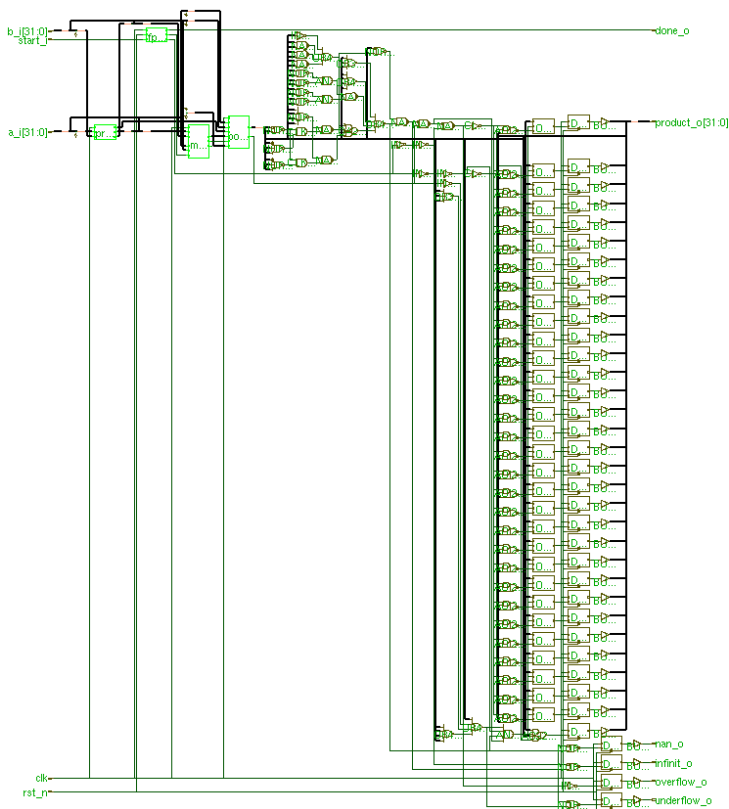
4.6. Síntese lógica e Layout

Aqui, é mostrada a síntese lógica (Figura 13) e o *layout* (Figuras 14 e 15) do circuito para as frequências de *clock* de 10MHz e 368MHz, respectivamente.

Figura 13: Resultados da síntese lógica do circuito na ferramenta *Genus*.



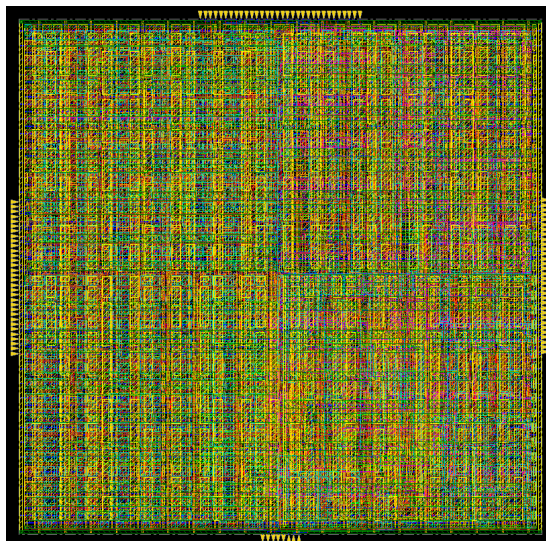
(a) 10 MHz



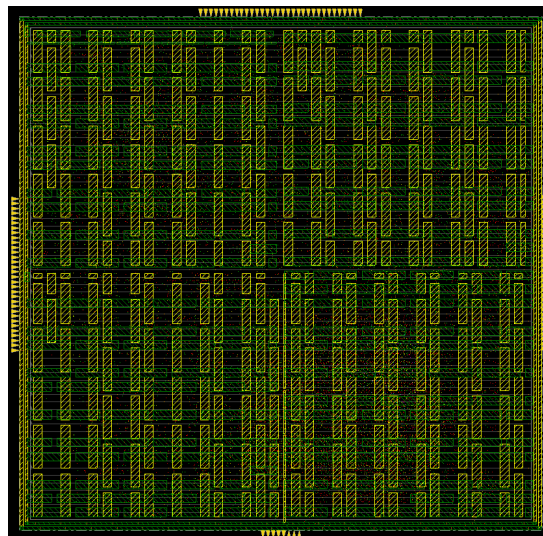
(b) 368 MHz

Fonte: Autor.

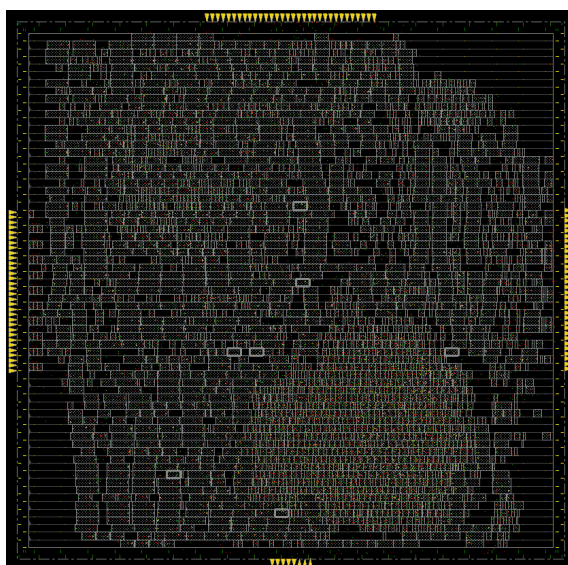
Figura 14: Diferentes visões do *Layout* para 10 MHz na ferramenta *Innovus*.



(a) *Layout* completo

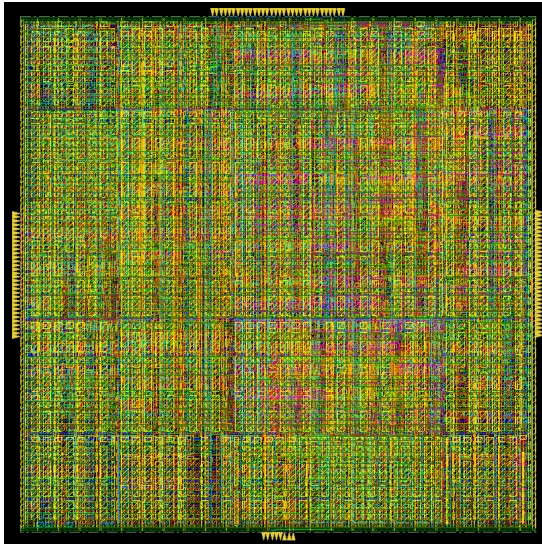


(b) Anéis e linhas de alimentação

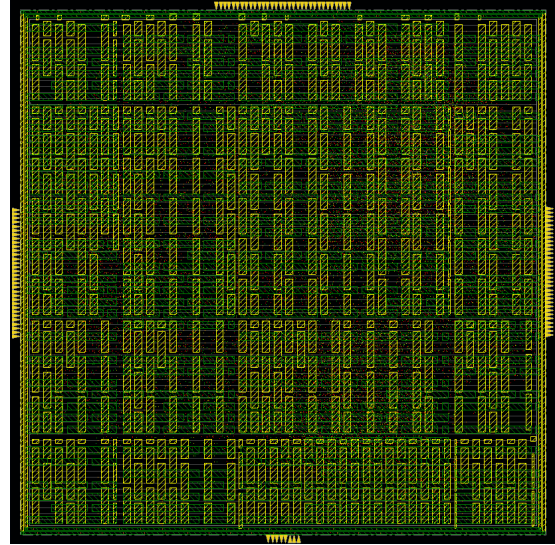


(c) *Standard Cells* sem *Filler*

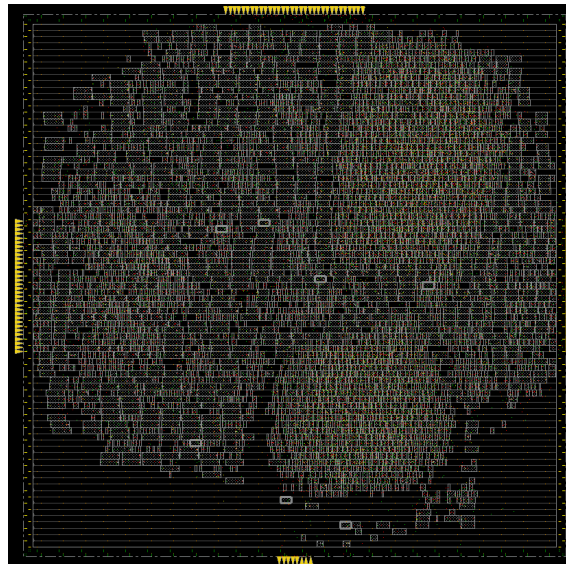
Figura 15: Diferentes visões do *Layout* para 368 MHz na ferramenta *Innovus*.



(a) *Layout* completo



(b) Anéis e linhas de alimentação



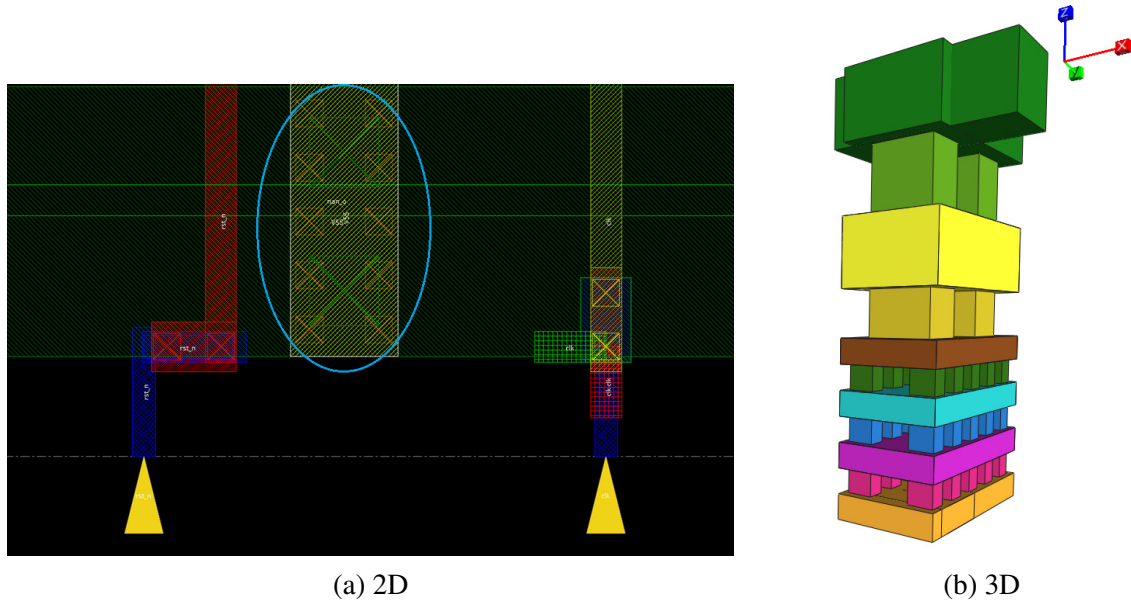
(c) *Standard Cells* sem *Filler*

Na Figura 14, podemos observar um espaçamento maior entre os pinos de entrada e saída relativo ao *layout* da Figura 15. Isso se deve ao maior tamanho do circuito como um todo para a frequência mais elevada.

Em ambos os *layouts*, temos uma região com alta densidade de células (visto nas Figuras 14c e 15c). Estas são em sua maioria parte do módulo de pós-normalização, que é o módulo com maior complexidade combinacional.

Além disso, na Figura 16, temos o detalhe (em 2D e 3D) de uma das linhas de alimentação, onde podemos observar um exemplo da construção física que a ferramenta realiza.

Figura 16: Resultados da síntese física de uma das linhas de alimentação na ferramenta *Innovus*.



Fonte: Autor.

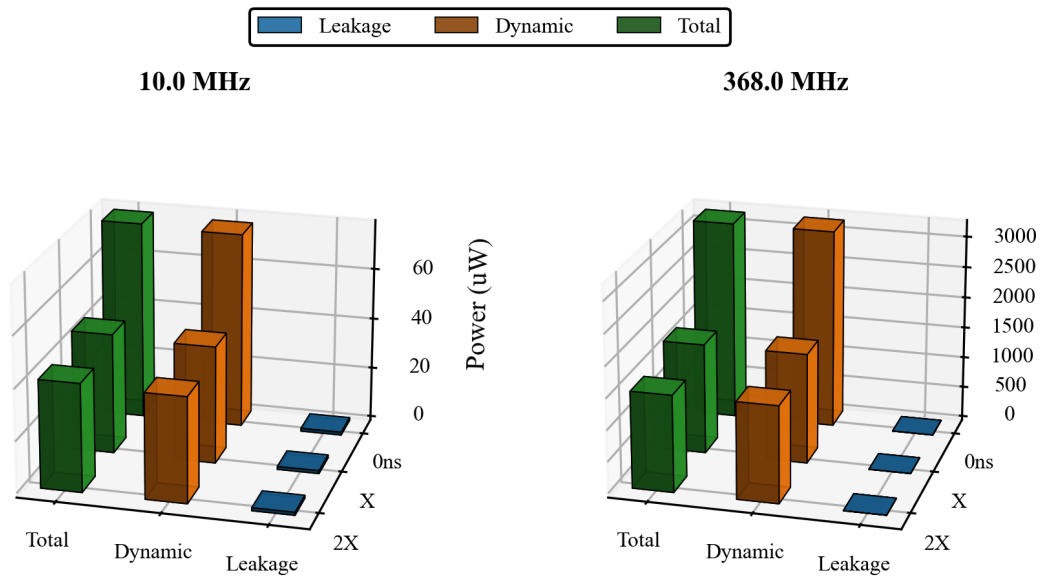
4.7. Métricas de *Timing* e Potência

Após realizar a síntese física do multiplicador, foram avaliadas as métricas de temporização e potência do circuito. Na Figura 17, podemos observar que as potências totais do circuito são bem maiores na frequência mais alta. Para a potência dinâmica, temos a equação 1, onde α é o fator de atividade, C a capacitância total, V a tensão e f a frequência.

$$P_{dyn} = \alpha \cdot C \cdot V^2 \cdot f \quad (1)$$

Dessa maneira, o aumento de frequência de 10 MHz para 368 MHz justifica o aumento de potência dinâmica observado (de 60 μW para 3200 μW), sendo a diferença além do fator linear explicada pelo aumento de capacitância total das células (a área cresceu de 12921 μm^2 para 20154 μm^2).

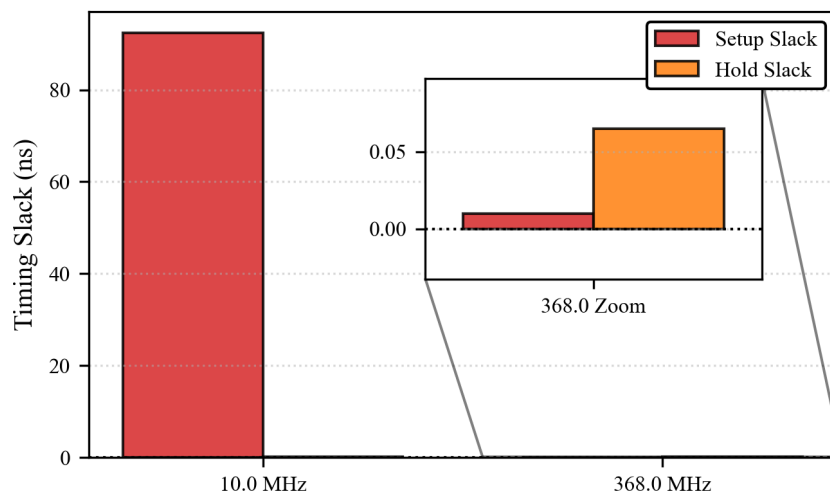
Figura 17: Gráfico da potência em relação à frequência e ao tempo de simulação (VCD).



Fonte: Autor.

Quanto ao *timing*, como a frequência máxima obtida possui *slack* próximo a zero, a Figura 18 reflete este comportamento.

Figura 18: Gráfico do *slack* em relação à frequência).



Fonte: Autor.

O caminho crítico identificado parte de `i_mul_24/count_reg[1]/CK` e termina em `i_mul_24/prod2_reg`, ou seja, está dentro do módulo `mul_24` (Estágio 2). Isso é esperado, pois a multiplicação de mantissas de 24 bits é apontada como o principal gargalo de desempenho em multiplicadores de ponto flutuante [2]. A alta densidade de células no módulo `post_norm_mul` (observada nas Figuras 14c e 15c) indica que esse módulo também contribui

significativamente para o caminho crítico, especialmente porque envolve lógica combinacional de normalização e arredondamento em cascata.

4.8. Comparação com a Literatura

Nesta subseção, foram comparados os parâmetros de área, potência e temporização do circuito multiplicador com as demais referências da literatura [5] [6]. Como as referências utilizam apenas a síntese lógica, foram utilizados os parâmetros referentes a esta para comparação. Na Tabela 3, estão dispostas as principais métricas comparadas.

Tabela 3: Parâmetros de Área e Potência em relação a literatura

Parâmetro	Este trabalho (368MHz)	Aditya [5] (Exato)	Aditya [5] (Aproximado)	Anuradha [6]
Frequência Máxima (MHz)	368	100	115.3	N/A
Delay (ns)	0.450	10.09	8.67	1.178
Área Total (μm^2)	19468.317	32515.619	7062.447	8834
Potência de <i>Leakage</i> (μW)	0.391	76.206	77.475	N/A
Potência Dinâmica (μW)	2360.369	11201.2	11054.5	N/A
Potência Total (μW)	2360.76	11277.3	11131.9	1619

Aqui, podemos ver que a potência consumida pelo multiplicador proposto por este trabalho é menor que as potências relativas ao multiplicador Védico exato e aproximado de Aditya et al. A potência de *leakage* (0.391 μW contra 76 μW [5]) reflete diretamente a diferença de tecnologia (o processo de Aditya é 90nm enquanto o deste trabalho é de 45nm), com transistores menores e menor corrente de fuga por célula. Porém, a abordagem aproximada de Aditya [5] apresenta uma área significativamente menor. Quanto ao multiplicador eficiente proposto por Anuradha et al. [6], tanto a área quanto a potência são menores. Essas diferenças se devem principalmente às diferenças de tecnologia, além de abordagens diferentes quanto ao problema de multiplicação de ponto flutuante, que pode gerar maior ou menor complexidade lógica.

5. CONCLUSÃO

Em suma, este trabalho apresentou o projeto e a implementação ASIC de um multiplicador em ponto flutuante de 32 bits seguindo o padrão IEEE 754, com arquitetura em pipeline de três estágios. O circuito foi desenvolvido em VHDL e validado por meio de simulações RTL, síntese lógica e síntese física, utilizando as ferramentas *Xcelium*, *Genus* e *Innovus* da *Cadence*.

A varredura de frequências de síntese permitiu determinar uma frequência máxima de

operação de 368 MHz, para a qual tanto a síntese lógica quanto o layout apresentam *timing slack* positivo. A área total do circuito nessa frequência foi de 20154.06 μm^2 , com potência total de aproximadamente 3202 μW e potência de *leakage* de apenas 1.396 μW .

As simulações demonstraram o correto funcionamento do circuito em todas as condições avaliadas, como *overflow*, *underflow*, infinito e *Not a Number*, confirmando a conformidade com as exceções previstas pelo padrão IEEE 754. A comparação entre as simulações RTL sem SDF, netlist com SDF pré-*layout* e pós-*layout* revelou o impacto progressivo das capacitâncias parasitas e atrasos reais sobre o comportamento temporal do circuito, evidenciando a importância da verificação em cada etapa do fluxo ASIC.

Em comparação com trabalhos da literatura, o multiplicador proposto apresentou potência dinâmica e total significativamente menores do que as implementações de Aditya et al. [5], embora com área superior à abordagem aproximada e à implementação de Anuradha et al [6]. Essas diferenças refletem tanto as distintas tecnologias de fabricação empregadas quanto as escolhas arquiteturais adotadas em cada trabalho.

Como trabalhos futuros, sugere-se a investigação de técnicas de redução de potência dinâmica (como *clock gating* e operandos com codificação de *Booth Radix-4*) e a exploração de tecnologias de processo mais avançadas, visando melhorar simultaneamente os parâmetros de área, potência e desempenho do multiplicador.

REFERÊNCIAS

- [1] G. Ushasree, R. Dhanabal, and S. K. Sahoo, “VLSI implementation of a high speed single precision floating point unit using Verilog,” in *Proceedings of the IEEE Conference on Information and Communication Technologies (ICT)*, pp. 803–808, 2013. IEEE.
- [2] N. Bai, H. Li, J. Lv, S. Yang, and Y. Xu, “Logic design and power optimization of floating-point multipliers,” *Computational Intelligence and Neuroscience*, vol. 2022, p. 6949846, 2022.
- [3] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, “Efficient acceleration of deep learning inference on resource-constrained edge devices: A review,” *Proceedings of the IEEE*, vol. 111, no. 1, pp. 42–91, 2023. IEEE.
- [4] A. Chopde and A. M. Kulkarni, “Physical design: Methodologies and developments,” *arXiv preprint arXiv:2409.04726*, 2024. Dept. of Electronics and Telecommunication Engineering, Vishwakarma Institute of Technology, Pune, India.

- [5] P. Aditya, B. Sankeerthana, K. S. Reddy, and P. V. S. Vasanth, “Asic implementation of approximate single precision floating point multiplier,” in *2023 World Conference on Communication & Computing (WCONF)*, pp. 1–4, 2023.
- [6] Anuradha, S. K. Patel, and S. K. Singhal, “An area-delay efficient single-precision floating-point multiplier for VLSI systems,” *Microprocessors and Microsystems*, vol. 98, p. 104798, 2023.
- [7] J. Al-Eryani, “32-bit Floating Point Unit (FPU).” <https://opencores.org/projects/fpu100>, 2006. Acessado em 18 de Maio de 2026.

Tabela 4: Dados de área, temporização e potência do multiplicador.

freq (MHz - clk)	area related data				worst case setup timing related data		power related data				
	cell count	net μm ²	total μm ²	equiv. gates	timing slack ps	Start point End point	Sim. Time (ns)		leak. (μW)	dynamic (μW)	total (μW)
10	2766	3565.122	12921.558	12594.111	87024	i_post_norm_mul/s_fract_48_i_reg[46]/CK i_post_norm_mul/s_zeros_reg[0]/D	—		0.208	40.449	40.657
							X	160551	0.208	21.77	21.978
							2X	321102	0.207	17.215	17.422
	5399	4214.124	13381.776	13042.667	92433	i_post_norm_mul/s_fract_48_i_reg[46]/CK i_post_norm_mul/s_zeros_reg[0]/D	—		1.153	76.583	77.736
							X	160551	1.15	45.6	46.75
							2X	321102	1.149	40.781	41.93
368	4380	5361.159	19468.317	18974.967	0	i_mul_24/count_reg[1]/CK i_mul_24/prod2_reg[3][2][16]/D	—		0.391	2360.369	2360.76
							X	4461	0.391	837.699	838.09
							2X	8922	0.389	649.188	649.578
	9982	9024.012	20154.060	19643.333	0.010	i_mul_24/count_reg[1]/CK i_mul_24/prod2_reg[0][3][10]/D	—		1.396	3200.991	3202.387
							X	4461	1.395	1760.373	1761.768
							2X	8922	1.393	1539.718	1541.111

Fonte: Autores.

Dados finais da síntese **lógica**

Dados finais da síntese **física**